

Functions and Parameters

Table of Contents

1. Introduction to Functions
2. Why Functions are Important
3. What is a Function?
4. Advantages of Functions
5. Types of Functions
6. Built-in Functions
7. User-Defined Functions
8. Syntax of Functions
9. Function Calling
10. Parameters and Arguments
11. Types of Parameters
12. Return Statement
13. Difference Between Print and Return
14. Scope of Variables
15. Nested Functions
16. Recursive Functions
17. Real-Life Examples of Functions
18. Common Errors in Functions
19. Practice Programs
20. Real-Life Applications
21. Conclusion

1. Introduction to Functions

As programs become larger, writing the same code repeatedly becomes difficult and inefficient.

To solve this problem, programming languages provide **functions**.

Functions help programmers:

- Reuse code
- Organize programs
- Reduce repetition
- Improve readability
- Make debugging easier

Functions are one of the most important concepts in programming.

2. Why Functions are Important

Functions help:

- Divide large programs into smaller parts
 - Reuse existing code
 - Improve program structure
 - Save development time
 - Make code easy to maintain
-

Example Without Functions

```
print("Welcome")
print("Welcome")
print("Welcome")
```

Example With Functions

```
def greet():
    print("Welcome")

greet()
greet()
greet()
```

3. What is a Function?

A **function** is a block of reusable code designed to perform a specific task.

Instead of writing the same code again and again, we can place it inside a function and call it whenever needed.

Real-Life Example

Think of a function like a machine.

Example:

- Coffee Machine → Makes coffee
 - Calculator → Performs calculations
 - Function → Performs a specific task
-

4. Advantages of Functions

Functions provide many benefits:

- Code reusability
 - Better readability
 - Easy debugging
 - Reduced code duplication
 - Better program organization
 - Faster development
-

5. Types of Functions

There are mainly two types of functions:

1. Built-in Functions
 2. User-Defined Functions
-

6. Built-in Functions

Built-in functions are functions already provided by Python.

Examples

Function Purpose

`print()` Displays output

Function Purpose

input() Takes user input
len() Finds length
type() Finds data type
int() Converts to integer
str() Converts to string

Example

```
name = "Python"  
  
print(len(name))
```

Output

6

7. User-Defined Functions

Functions created by programmers are called user-defined functions.

Syntax of Function

```
def function_name():  
    statements
```

Example

```
def greet():  
    print("Hello Students")
```

Function Calling

A function executes only when it is called.

Example

```
def greet():  
    print("Hello Students")
```

```
greet()
```

Output

Hello Students

Explanation

- def keyword is used to create function
 - greet is function name
 - () stores parameters
 - Indented block contains function body
 - greet() calls the function
-

8. Parameters and Arguments

What are Parameters?

Parameters are variables written inside function definition.

They help functions receive values.

Syntax

```
def function_name(parameter):  
    statements
```

Example

```
def greet(name):  
    print("Hello", name)  
  
greet("Rahul")
```

Output

Hello Rahul

Explanation

- name is parameter
 - "Rahul" is argument
 - Argument value is passed into parameter
-

Difference Between Parameter and Argument

Parameter	Argument
Variable in function definition	Actual value passed
Receives data	Sends data

Example

```
def add(a, b):  
    print(a + b)
```

```
add(10, 20)
```

Output

```
30
```

Here:

- a and b → Parameters
 - 10 and 20 → Arguments
-

9. Types of Parameters

9.1 Positional Parameters

Arguments are passed according to position.

Example

```
def student(name, age):  
    print(name, age)
```

```
student("Aman", 20)
```

9.2 Default Parameters

Default value is assigned to parameter.

Example

```
def country(name="India"):  
    print(name)
```

```
country()  
country("USA")
```

Output

```
India  
USA
```

9.3 Keyword Arguments

Arguments are passed using parameter names.

Example

```
def details(name, age):  
    print(name, age)
```

```
details(age=20, name="Rahul")
```

9.4 Variable Length Arguments

Used when number of arguments is unknown.

Example

```
def total(*numbers):  
    print(sum(numbers))
```

```
total(1, 2, 3, 4)
```

Output

10

10. Return Statement

The return statement sends a value back from function.

Example

```
def add(a, b):  
    return a + b
```

```
result = add(5, 3)
```

```
print(result)
```

Output

8

Importance of Return

- Sends output back
 - Allows result reuse
 - Makes functions more powerful
-

11. Difference Between print() and return

print()

Displays output

Cannot reuse directly

return

Sends value back

Can store in variable

print()

Used for display

return

Used for processing

Example

```
def square(num):  
    return num * num
```

```
value = square(4)
```

```
print(value)
```

Output

16

12. Scope of Variables

Scope means where a variable can be used.

Local Variable

Created inside function.

Example

```
def test():  
    x = 10  
    print(x)
```

```
test()
```

Global Variable

Created outside function.

Example

```
x = 100
```

```
def show():  
    print(x)
```

```
show()
```

13. Nested Functions

A function inside another function is called nested function.

Example

```
def outer():
```

```
    def inner():  
        print("Inner Function")
```

```
    inner()
```

```
outer()
```

14. Recursive Functions

A function calling itself is called recursion.

Example

```
def countdown(n):
```

```
    if n == 0:  
        return
```

```
    print(n)
```

```
    countdown(n - 1)
```

```
countdown(5)
```

Output

5
4
3
2
1

15. Real-Life Examples of Functions

Functions are used in:

- ATM systems
 - Banking software
 - Mobile applications
 - AI systems
 - Websites
 - Games
 - Data analysis software
-

16. Common Errors in Functions

1. Forgetting Function Call

Wrong:

```
def greet():  
    print("Hello")
```

No output because function is not called.

Correct

```
greet()
```

2. Missing Parameters

```
def add(a, b):  
    print(a + b)
```

```
add(5)
```

This causes error because one argument is missing.

3. Wrong Indentation

Python requires proper indentation inside functions.

17. Practice Programs

Program 1: Addition Function

```
def add(a, b):  
    return a + b  
  
print(add(10, 20))
```

Program 2: Even or Odd Function

```
def check(num):  
  
    if num % 2 == 0:  
        return "Even"  
  
    return "Odd"  
  
print(check(5))
```

Program 3: Square Function

```
def square(num):  
    return num * num  
  
print(square(4))
```

Program 4: Greeting Function

```
def greet(name):  
    print("Welcome", name)  
  
greet("InternStudio")
```

18. Real-Life Applications

Functions are widely used in:

- Login systems
- Payment systems
- AI models
- Search engines
- Web development
- Data science
- Automation software